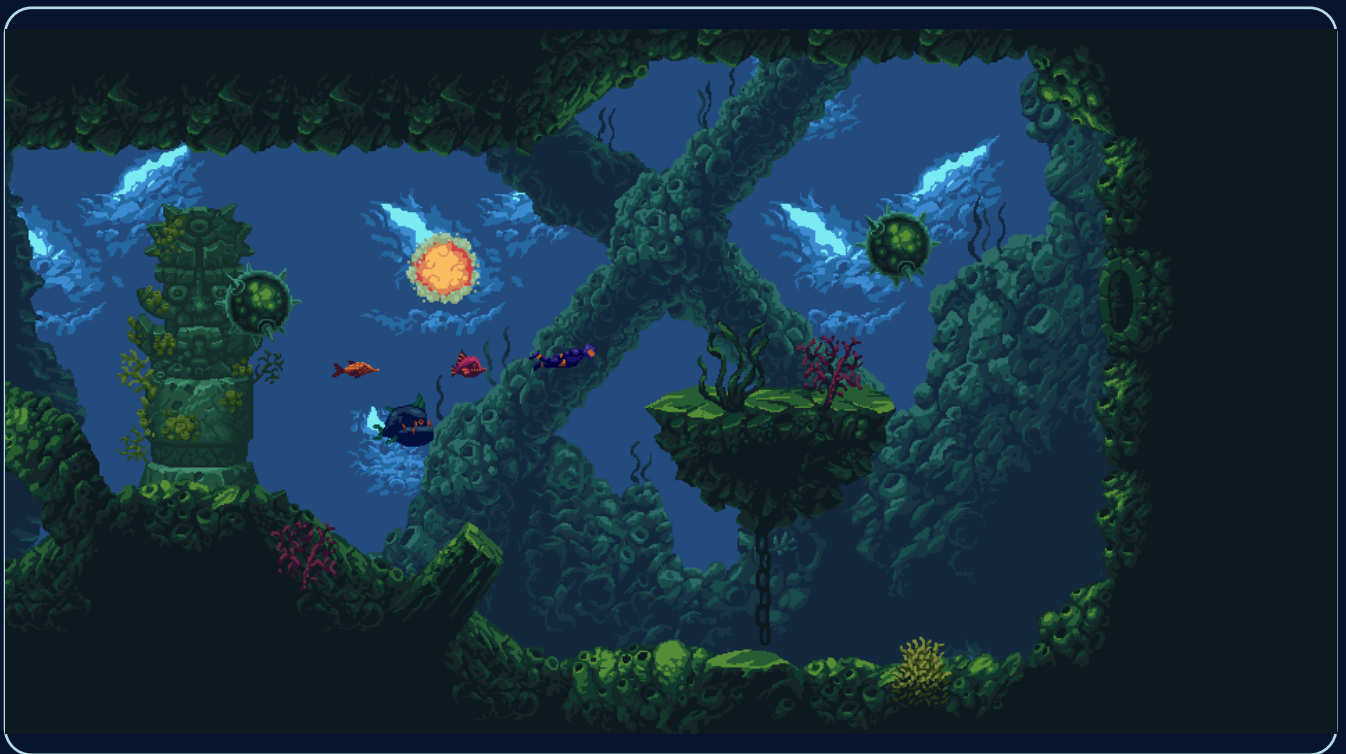


MAREA ALTA

Informe completo del proyecto

Estructura, personajes, assets, escenarios, código y propósito ambiental



DEMO 2D · UNITY 6.3 LTS · PC

Explicado con palabras simples y ejemplos visuales

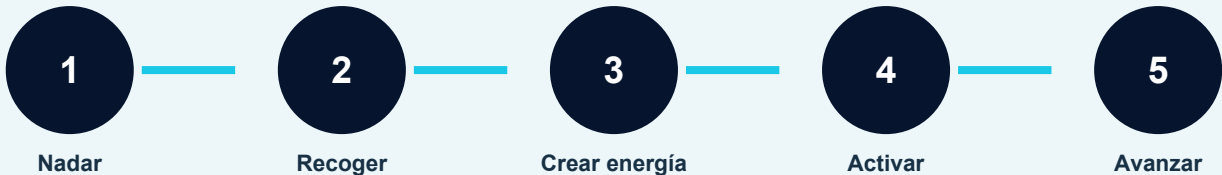
¿Qué es Marea Alta?

Una aventura para ayudar al océano

Marea Alta es un videojuego 2D en el que controlamos a Mara, una buzo que entra en un arrecife contaminado. Su misión es recoger basura, convertirla en energía y encender turbinas para abrir nuevas zonas.



EL CICLO DEL JUEGO



Contenido del informe

- Cómo se ordenó el proyecto dentro de Unity.
- Cómo se creó a Mara y cómo se mueve.
- Qué imágenes, sonidos y objetos se utilizan.
- Cómo se construyeron las tres zonas del nivel.
- Qué hace el código y cuáles son las partes más importantes.
- Cómo se relaciona el juego con los ODS 7 y 14.

Cómo está organizado el proyecto

El proyecto funciona como una caja de herramientas bien ordenada. Cada carpeta guarda un tipo de pieza para encontrarla rápidamente.

CARPETAS PRINCIPALES

/Scenes	guarda el menú y el nivel
/Scripts	contiene las reglas del juego
/Art	guarda personajes y objetos
/Audio	música y efectos
/Prefabs	objetos listos para reutilizar
/UI	textos, botones y avisos

ESCENAS

MainMenu

Pantalla de inicio.

Prototype_PuntaCoral

Respaldo del primer prototipo.

Demo_PuntaCoral

Versión ampliada y presentable.

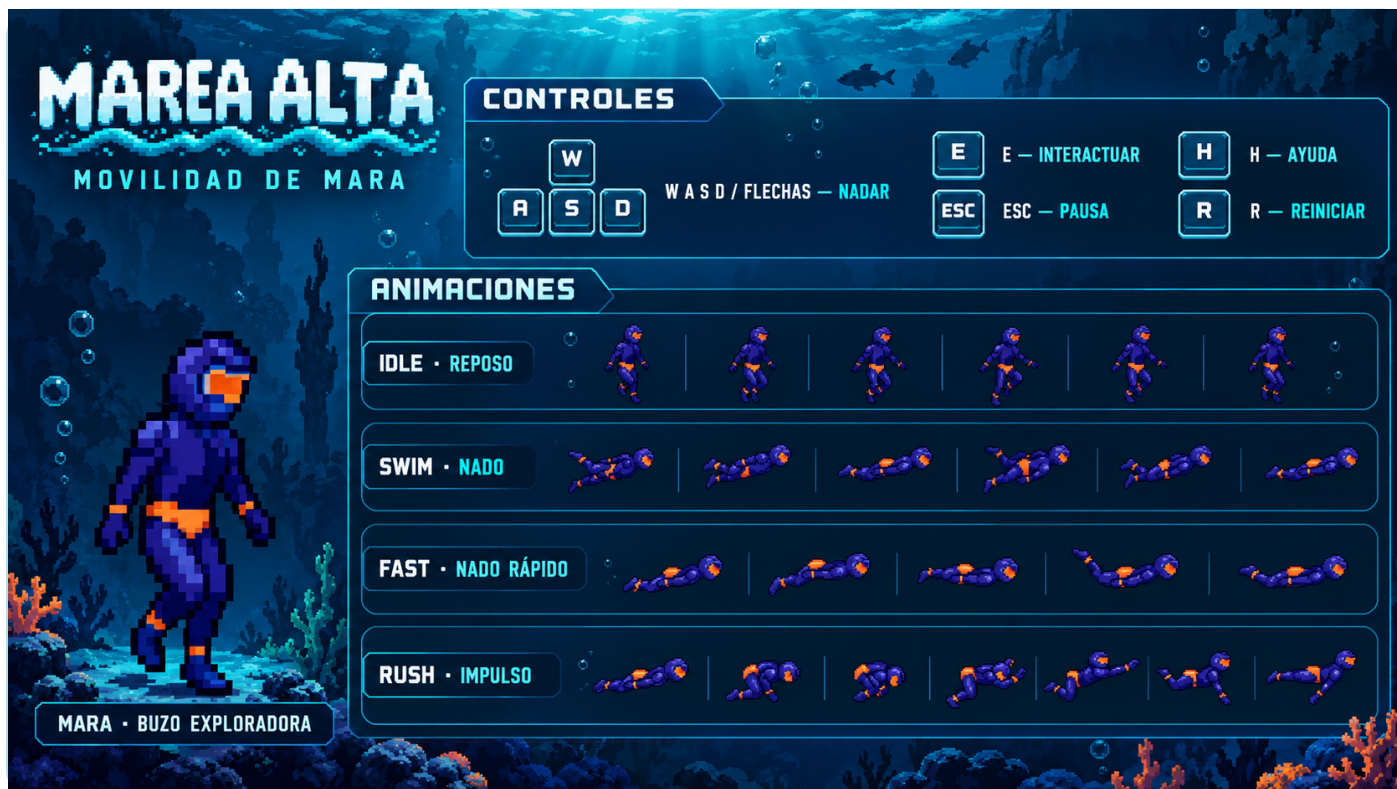
Ejemplo fácil

Una escena es como una página de un cuento. El menú es una página y el arrecife es otra. Los scripts son las instrucciones que dicen qué puede ocurrir en cada página.

Tecnología utilizada

- Unity 6.3 LTS con Universal 2D.
- Lenguaje C# para las reglas.
- Rigidbody2D y Collider2D para movimiento y choques.
- Animator para cambiar entre reposo y nado.

Mara: personaje y movimiento



¿Cómo se creó?

- Se importó el personaje del paquete Pixel Art Diver.
- La hoja de sprites se cortó en cuadros, como las páginas de un pequeño libro animado.
- Los cuadros Idle muestran a Mara quieta. Los cuadros Swim muestran a Mara nadando.
- El Animator cambia de animación según la velocidad del personaje.

Controles actuales

WASD o flechas: nadar · E: usar una turbina · Esc: pausa · H: ayuda · R: reiniciar después de ganar.

Assets: las piezas visuales y sonoras



Un asset es una pieza que el juego utiliza: una imagen, un sonido, una animación o un objeto preparado.

ASSETS UTILIZADOS

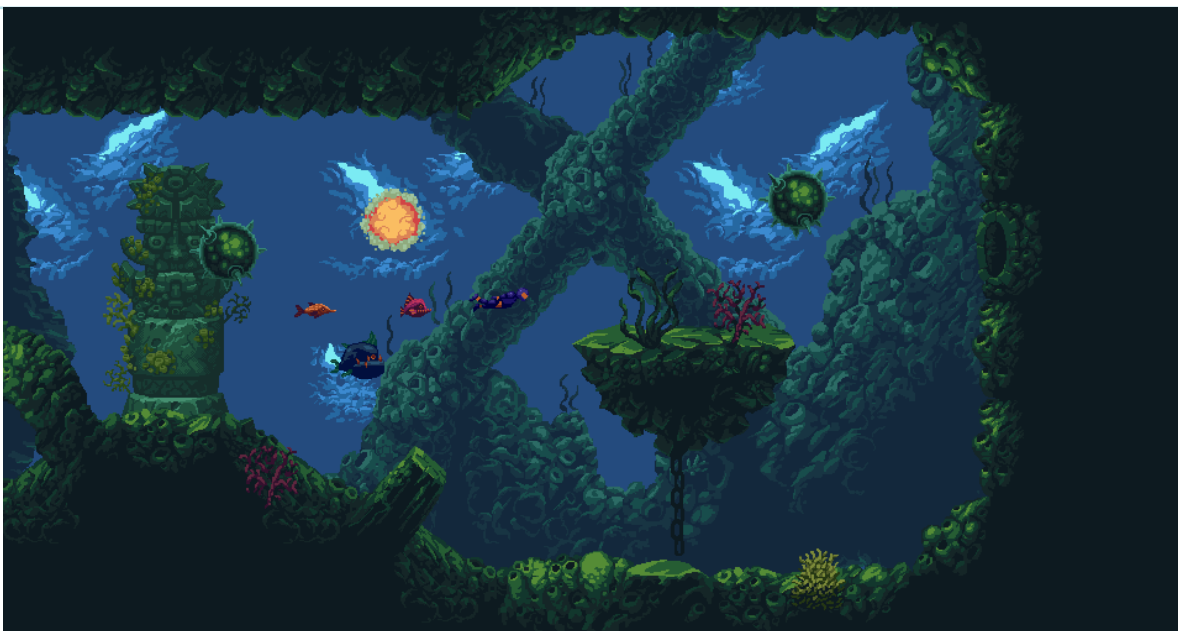
- Mara y sus animaciones.
- Fondos, arrecifes, tiles y plantas.
- Residuos, turbinas y compuertas.
- Peces, burbujas, música y efectos.

ASSETS PREPARADOS

- Logotipo e icono del juego.
- Pez Dardo y Pez Abisal.
- Mina Coralina.
- Leviatán de Residuos.

Los enemigos son conceptos visuales: todavía no tienen comportamiento dentro de la demo.

Cómo se construyó el escenario



El nivel se divide en tres partes

1

Entrada del arrecife

Aprender a nadar, recoger tres residuos y encender la primera turbina.

2

Ruinas hundidas

Cruzar la compuerta, buscar otros tres residuos y activar la segunda turbina.

3

Fosa de salida

Recoger los últimos residuos, abrir la compuerta final y llegar a la baliza.

Las paredes usan Collider2D: son como barreras invisibles colocadas exactamente encima de las rocas. La compuerta también tiene una barrera, pero el código la desactiva cuando se abre.

Las reglas principales del juego

RECOLECCIÓN

Mara toca un residuo. El objeto suma un fragmento y desaparece para que no pueda recogerse dos veces.

ENERGÍA

Cada grupo de tres fragmentos se convierte automáticamente en una unidad de energía.

TURBINA

Al pulsar E, la turbina pregunta si Mara tiene energía. Si la tiene, la usa una sola vez.

COMPUERTA

La puerta sube, se hace pequeña y apaga sus colisionadores para permitir el paso.

SALIDA

La baliza detecta a Mara, detiene el movimiento y muestra el resultado de la partida.

¿Qué ocurre si no hay energía?

La turbina no se activa. El HUD explica cuánta energía falta y suena un aviso de error. Así el jugador entiende qué debe hacer sin perder el progreso.

Código explicado: movimiento y energía

Mover a Mara

```
if (W) input.y += 1;  
if (S) input.y -= 1;  
if (A) input.x -= 1;  
if (D) input.x += 1;  
  
velocity = MoveTowards(  
    velocity, target, speed);
```

El programa mira qué teclas están presionadas. Después calcula hacia dónde debe nadar Mara. MoveTowards hace que acelere y frene suavemente, como si estuviera dentro del agua.

Crear energía

```
total = fragments + amount;  
energy = total / 3;  
fragments = total % 3;  
  
EnergyCount += energy;
```

La división responde cuántas energías completas se pueden crear. El símbolo % guarda los fragmentos sobrantes. Por ejemplo: 5 fragmentos crean 1 energía y dejan 2 fragmentos.

La parte más importante

No se utiliza una fórmula demasiado difícil. Lo especial es que varios scripts trabajan juntos sin confundirse: movimiento, inventario, sonidos, animaciones y HUD comparten el estado correcto.

Código explicado: turbina y compuerta

```
if (inventory.TrySpendEnergy(1))  
{  
    IsActivated = true;  
    linkedGate.Open();  
}
```

La turbina intenta gastar una energía. Si puede hacerlo, cambia a encendida y manda una orden a la compuerta: Abrir.

```
gateCollider.isTrigger = true;  
gateCollider.enabled = false;
```

Cuando la puerta está abierta, su barrera se apaga. Esta corrección fue necesaria porque antes la puerta se veía abierta, pero todavía bloqueaba a Mara.

OTROS SCRIPTS IMPORTANTES

- GameAudio reproduce música y sonidos desde un solo lugar.
- CameraFollow2D sigue a Mara sin salir de los límites del nivel.
- PrototypeHUD muestra fragmentos, energía, zona y objetivo.
- PauseController congela el tiempo y ofrece continuar, reiniciar o volver al menú.
- GameSession guarda el tiempo, detecta la victoria y permite reiniciar con R.

Interfaz, sonido y pruebas

INTERFAZ

- Menú principal con identidad propia.
- HUD con recursos y progreso.
- Avisos para usar turbinas.
- Pausa, victoria y resultados.

SONIDO

- Música ambiental continua.
- Sonido al recoger basura.
- Avisos de energía y error.
- Turbina, compuerta, victoria y botones.

Pruebas realizadas

- Los nueve residuos se recogen una sola vez.
- Tres residuos producen una energía.
- Cada turbina abre su compuerta y cambia de color.
- Los arrecifes bloquean a Mara y las puertas abiertas dejan pasar.
- El recorrido completo de las tres zonas termina en la baliza.
- La pausa, el reinicio y el regreso al menú funcionan.

Una prueba sirve para comprobar que el juego hace lo esperado. Cuando algo falla, se corrige y se vuelve a probar. Así se arreglaron el tamaño de los residuos y el colisionador de la compuerta.

ODS 7 y ODS 14

ODS 14 · VIDA SUBMARINA

- El problema principal es la basura en el océano.
- Mara limpia el arrecife y protege su hábitat.
- Los peces y corales recuerdan que el mar está lleno de vida.
- La futura restauración visual mostrará cómo mejora el ecosistema.

ODS 7 · ENERGÍA LIMPIA

- Los residuos recuperados se convierten simbólicamente en energía.
- La energía activa turbinas y recupera la infraestructura.
- La mecánica habla de reutilizar materiales y cuidar los recursos.
- Es una idea educativa; no afirma que todo plástico produzca energía limpia directamente.

Los dos objetivos trabajan juntos

LIMPIAR → RECUPERAR → GENERAR → RESTAURAR

El ODS 14 explica por qué Mara quiere salvar el arrecife. El ODS 7 explica cómo la energía ayuda a abrir caminos y devolver el funcionamiento a Punta Coral.

Enemigos y expansión futura



Los siguientes diseños ya tienen imágenes y hojas de sprites, pero todavía no tienen inteligencia, daño ni combate dentro del prototipo.

- **Pez Dardo:** prepararía una carga rápida.
- **Pez Abisal:** podría patrullar y perseguir a Mara.
- **Mina Coralina:** mostraría una luz antes de activarse.
- **Leviatán de Residuos:** sería el jefe final y tendría varias fases.

Otras mejoras previstas

Dash, corrientes marinas, marea dinámica, controles móviles, restauración progresiva del arrecife, fauna reactiva y música adaptativa.

Estado actual y conclusión

Lo que ya está terminado

- Proyecto Unity 2D organizado y funcional.
- Mara puede nadar, animarse y chocar con el escenario.
- Nueve residuos, tres turbinas, tres compuertas y tres zonas.
- Conversión automática de fragmentos en energía.
- Menú, HUD, pausa, audio, victoria y resultados.
- Material visual para presentación, logotipo e icono de aplicación.

Lo que sigue después

- Terminar el rediseño profesional de la interfaz.
- Tomar capturas finales y montar la presentación.
- Implementar enemigos solo si el tiempo del proyecto lo permite.
- Añadir marea, corrientes, restauración y controles móviles en futuras versiones.

Conclusión

Marea Alta ya es una demo corta, clara y jugable. Su mayor fortaleza es que cada acción tiene un propósito fácil de entender: limpiar, crear energía y abrir el camino para recuperar el arrecife. El código no necesita trucos imposibles; funciona porque cada pieza tiene una tarea pequeña y todas colaboran.

Créditos y archivos principales

Recursos visuales

- Personaje y recursos submarinos: Underwater Diving Pack.
- Objetos propios de la demo: residuos, turbina y compuerta generados para Marea Alta.
- Logotipo, icono, bestiario y láminas preparados durante el desarrollo del proyecto.

Audio

La música y los efectos utilizados son recursos CC0. Sus autores, enlaces y licencias están registrados dentro del proyecto en:

```
Assets/ThirdParty/AudioCC0/  
SOURCES_AND_LICENSES.md
```

Scripts principales

SwimController2D · EnergyInventory · DebrisCollectible · PlayerInteractor · TurbineController · GateController · CameraFollow2D · PrototypeHUD · GameAudio · GameSession

Idea final

Construir un videojuego se parece a armar un rompecabezas. Las imágenes muestran el mundo, los sonidos le dan vida y el código decide cómo responden las piezas cuando el jugador actúa.